

ARC ROS2 Workshop

```
codium . --no-sandbox
```

Sub Cmd_vel

```
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

class CmdVelReader(Node):
    def __init__(self):
        super().__init__('cmd_vel_reader')

        self.subscription = self.create_subscription(
            Twist,
            '/cmd_vel',
            self.cmd_vel_callback,
            10
        )

        self.get_logger().info('Listening to /cmd_vel...')

    def cmd_vel_callback(self, msg: Twist):
        self.get_logger().info(
            f'linear: x={msg.linear.x:.3f}, '
            f'y={msg.linear.y:.3f}, '
            f'z={msg.linear.z:.3f} | '
            f'angular: x={msg.angular.x:.3f}, '
            f'y={msg.angular.y:.3f}, '
            f'z={msg.angular.z:.3f}'
        )

def main(args=None):
    rclpy.init(args=args)

    node = CmdVelReader()

    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass

    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

sub scan

```
#!/usr/bin/env python3

import math

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan

class ScanDirectionReader(Node):
    def __init__(self):
        super().__init__('scan_direction_reader')

        self.subscription = self.create_subscription(
            LaserScan,
            '/scan',
            self.scan_callback,
            10
        )

        self.get_logger().info('Listening to /scan...')

    def scan_callback(self, scan: LaserScan):
        front_index = int(round((0.0 - scan.angle_min) / scan.angle_increment))
        left_index = int(round((math.radians(90) - scan.angle_min) / scan.angle_increment))
        right_index = int(round((math.radians(270) - scan.angle_min) / scan.angle_increment))
        behind_index = int(round((math.radians(180) - scan.angle_min) / scan.angle_increment))

        front = scan.ranges[front_index]
        left = scan.ranges[left_index]
        right = scan.ranges[right_index]
        behind = scan.ranges[behind_index]

        self.get_logger().info(
            f'front={front:.3f} m | '
            f'left={left:.3f} m | '
            f'right={right:.3f} m | '
            f'behind={behind:.3f} m'
        )

def main(args=None):
    rclpy.init(args=args)

    node = ScanDirectionReader()

    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass

    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Stop the robot

```
#!/usr/bin/env python3
```

```
import math
```

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan
from geometry_msgs.msg import Twist
```

```
class StopOnObstacle(Node):
    def __init__(self):
        super().__init__('stop_on_obstacle')

        self.stop_distance = 0.3 # meters

        self.scan_subscriber = self.create_subscription(
            LaserScan,
            '/scan',
            self.scan_callback,
            10
        )

        self.cmd_vel_publisher = self.create_publisher(
            Twist,
            '/cmd_vel',
            200
        )

        self.get_logger().info('Stop-on-obstacle node started.')

    def scan_callback(self, scan: LaserScan):
        front_index = int(round((0.0 - scan.angle_min) / scan.angle_increment))
        front_distance = scan.ranges[front_index]

        if front_distance < self.stop_distance:
            stop_msg = Twist()
            stop_msg.linear.x = 0.0
            stop_msg.linear.y = 0.0
            stop_msg.linear.z = 0.0
            stop_msg.angular.x = 0.0
            stop_msg.angular.y = 0.0
            stop_msg.angular.z = 0.0

            self.cmd_vel_publisher.publish(stop_msg)

            self.get_logger().warn(
                f'Obstacle in front: {front_distance:.3f} m. Stopping robot.'
            )
        else:
            self.get_logger().info(
                f'Front clear: {front_distance:.3f} m'
            )
```

```
def main(args=None):
    rclpy.init(args=args)

    node = StopOnObstacle()

    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass

    node.destroy_node()
    rclpy.shutdown()
```

```
if __name__ == '__main__':
    main()
```